Episode Event Stream build — bugs and lessons

Oct 16, 2026 · @Vijay

How to read this

Most of the event stream's bugs come from about a dozen habits, not from hard problems. Each one could have been caught by a quick look at the data, a question to Vijay, or a test on the step's own output. The fixes are already in hand. This doc asks a different question: why did each one get in, and what would have stopped it?

What was reviewed: notebook `04 - Populate Episode Event Stream` (404 cells, the 12 October version), the table `ods.episodeeventstream` as built on 24 September 2026, and the flow sheets in `Chill_Episode_Frameworks.xlsx`.

- **Reviewed and agreed with Vijay:** all seven Motor journeys and Home Acquisition.
- **Reviewed, questions still open:** Home Renewal onwards, Van and Travel.

The full step-by-step log is `out/e1_findings.md`. The open questions are in E1 Episode build — open questions.

Every bug below carries one of four tags, naming what would have caught it:

Tag	What it means
Check the data	The answer was in the source. Sampling rows, profiling a column or counting first would have shown it
Ask Vijay	The step needed business knowledge: which source is trusted, what a code means, how the process really runs
Test the output	The step's own rows show the bug: duplicates, zero rows, dates outside the window, two steps with the same count
Read the design	The flow sheet already said what the step measures, or that it has no signal

For each bug, think about two things. Which of these would you have done? And at what point in writing the cell?

Summary

The 122 findings on the reviewed steps come down to 15 patterns. Nearly all of them would have been caught by one check before handover.

The largest group is steps that measure something other than what the step describes: 47 findings. That is a conversation with Vijay, or a careful read of the flow sheet, that did not happen. Next come habits that a test on the step's own output would have caught: wrong windows, duplicate rows, identical steps and zero-row cells. The stream as built has 52.9M rows across 316 steps.

Pattern	What goes wrong	Findings	What would have caught it
P9 Source does not measure the step	The filter picks something other than what the step describes	47	Ask Vijay, read the design
P3 Proxy timestamp	The time is a sale date, due date or due − 60, not when the step happened	25	Check the data
P2 Copy-paste step	Two steps insert the same rows, or a description is copied from another step	23	Test the output
P7 Window not applied	Events from 2015, or from 2027, get in. Literal dates are left over from count queries	22	Test the output
P12 No episode anchor	An event is joined to the policy or customer, not to the cycle or sale it belongs to	20	Read the design
P5 Wrong key column	An email, phone or guid sits in a customer or policy column	15	Check the data
P15 Whole-window collapse	`GROUP BY PolicyCode` with no date: one event per policy over 5 years	15	Test the output
P8 Hidden dependency	A table built by another notebook, a scratch notebook or a one-off copy	14	Ask Vijay
P4 Mirrored code	Copied from another product, keeping its filters and dates	12	Check the data

Pattern	What goes wrong	Findings	What would have caught it
P6 Fan-out repeats	A join to a one-to-many source creates identical rows	9	Test the output
P10 Call selection	Only the last session's queue, inbound only, or phone matching with no time limit	7	Check the data
P11 Literal bug	SQL that cannot do what it says (string compared as a column, NOT IN with a NULL)	7	Test the output
P1 Not re-runnable	Cells append, so a re-run duplicates rows. Hand patches on build day	5	Test the output
P14 Month bucketing	One event per calendar month, keeping the latest time	4	Ask Vijay
P13 Mixed policy key	Some steps key on last year's code, others on this year's	3	Check the data

One finding can carry several patterns, so the counts add up to more than 122.

The five that matter most

1. **97% of rows have no customer key.** 305 of 316 steps never fill it, 95% of rows have no event reference, and 44% have no policy. Without keys the stream cannot form a customer's episode at all. That was the stream's purpose, so the gap should have been visible on the first test.
2. **Steps that count the wrong thing (P9).** "Renews on an inbound call" (R3b) is every renewal posting through any channel: 241,733 of its 374,778 policies also renewed online. "Renews on an outbound call" (R3c) uses the inbound channel code.
3. **Invented times (P3).** Chases are stamped at the sale date and outcomes at the due date. The window opening is stamped at due − 60. The order of events inside an episode is therefore not real.
4. **Events with no episode (P12).** "Shops at renewal" (R2.B1.F1) takes every quote by the customer in 5 years: 313,077 of 386,673 are more than a year from any due date. "Policy lapsed" (R3.O1) writes the current status on every renewal: 446,153 of its 613,403 rows renewed on time.
5. **Counts inflated by re-runs and duplicates (P1, P2, P6).** A6.B1.F1 was written 3 times and A6.F1 twice. A7.B1 and A7.B1.1 insert identical rows. A6.B1.3 has up to 146 copies of one row.

Foundations: sources, keys, clocks and windows

These bugs sit under every journey. A step built on the wrong source, key or window is wrong however good its logic. All of them were knowable before writing the step: a source question to Vijay, or a five-minute profile of the table.

Bug	Example and evidence	Why it got in	What should have been done
Hidden dependencies (P8)	04 reads tables it never builds. `stg.A3_SL10_QQGUIDS` is a one-off copy of the on-prem SL_10 report, so A3 starts in 2024 and can never refresh. Also: `stg.MFQ_Quote_Payments` (notebook 05), `ods.EventStream` (notebook 10, stops 2026-09-14), `stg.motorclaims`, `tmp.CurrentPolicy`, and `stg.VM0_van_phone_numbers` (scratch notebook, pii schema)	A table that existed in the lakehouse was taken as a source. Nobody checked who builds it, or whether it would be there next time	List every FROM table, and find the notebook that creates each one. **Ask Vijay** which are sources of record and which are one-offs. Agreed rule now: one notebook builds everything it reads (R-2)
A source already known to be wrong	Arrears and lapse cells read `ods.scvcampaignhistory`. It is confirmed wrong; `scvcampaignhistorynew` is the right one	Two tables with nearly the same name, and the first one found was used	**Ask Vijay** when two tables look alike. Write the answer down where the next person will see it
Wrong key inside the source	In `edw.tbl_fact_policy_mtc` the cancelled policy is `OldPolicyCode`; `PolicyCode` is the new CXL record. Joining on PolicyCode left C1a with 113 of 2,389 customer cancellations	The column name was trusted, not the data	**Check the data:** follow one known cancellation through the table before writing the join
Flags and dates that don't mean what they say	`RecordVersionIsLatest` is unreliable in `tbl_ref_policy` (use MAX(RecordVersion)). `PolicyCancelDate` is filled on lapses too, with junk years up to 7709	A flag called "latest" was taken at its word	**Check the data:** count rows per policy where the flag is 1. Profile the min and max of every date column used
A key the product does not have	Home sales never carry a QuoteQueryGUID (0 of 87,387 since 2021-07). So HA4b "agent converts on the call" counts every Home sale, and HA4.F1 is never true. The Home link is HEQReference	Motor logic copied to Home without checking the column was filled	**Check the data:** count non-null values of the join column for that product before mirroring

Bug	Example and evidence	Why it got in	What should have been done
Wrong thing in a key column (P5)	An email address in SourceCustomerReference (A2.E1, A5.1). A quote guid in SourcePolicyReference (A3.B1.1). Phone numbers as customer references (A3.E1, D.V1)	The column was used to carry whatever key the cell had	**Test the output:** sample 10 rows per step and read every reference column. Only an SCV key goes in the customer column; anything else stays blank (R-6)
Customers found by phone number with no time limit	A0 "no MFQ match" means the caller's phone never appears on any quote in 5 years. M1 drops a Motor caller whose phone is on a Van list, so a customer with both is lost	A phone list was the quickest link to hand	**Ask Vijay** for the agreed customer link (`ods.customercallevents`). If phone matching is ever needed, limit it to a window around the call
Two clocks mixed	MFQ `CreatedDateTime` was Irish local time until 2026-06-11 and UTC from 2026-06-12. Genesys, Xtremepush, payments and HFQ are UTC. So in summer, MFQ events sit up to an hour away from the call or payment they relate to	Timestamps were assumed to share one clock	**Check the data:** compare a quote's last edit with its payment time, by month. A one-hour summer shift shows straight away (found in RT1)
GDPR-wiped values used as data	Up to 2026-05-06, email, mobile and registration are overwritten with `GDPR` on non-consent, non-buying quotes. A0 matches phone numbers against quotes where the mobile is `GDPR`	The sentinel looks like a value	**Check the data:** list the top 10 values of every key column. `GDPR` sits at the top
Window not applied (P7)	R0 and A4b read renewals and sales from 2015 (A4b has 205,703 rows before the window). The renewal base runs to 2027-08-31 (158,254 rows due after the window). Literal dates such as `2026-07-01 .. 2026-08-10` and end dates of 2026-08-10, 2026-08-01 and 2026-07-31 are left over from one-off count queries	Window filters were added cell by cell, and test dates were never taken out	**Test the output:** min and max timestamp per step against the build window. Take the window from the build config only, never as a literal

Event stream and episode logic

An episode is a sequence: the right events, for the right customer and cycle, in the right order. The bugs here break one of those three. Most show up the moment you put one customer's events side by side and read them in time order. That test was never run.

Bug	Example and evidence	Why it got in	What should have been done
Not re-runnable (P1)	Every cell appends. On 2026-09-24, A6.B1.F1 was written 3 times, A6.F1, A7.F1 and M.V3 twice. A3 and A3.F1 were patched by hand (DELETE and rewrite at 18:39)	Cells were re-run to fix errors, and the earlier rows stayed	Design each step to delete its own rows (or rebuild its table) before inserting. **Test the output:** run the notebook twice; the counts must not change. Delta history shows two writes with the same row count
Invented times (P3)	Chases stamped at the sale date (A6, HA6) while the chase snapshot carries the send date. R0 at due − 60. R4 "payment taken" at the due date, though the portal source has a Timestamp. A3 at quote start, not when the price was shown	When no time was obvious, a nearby date was used, with nothing on the row to say so	**Check the data:** does the source hold the real time? Usually it did. If it really doesn't, **ask Vijay**, and mark the row as derived (proposed `EventTimeBasis` column, X5)
Events with no episode anchor (P12)	R2.B1.F1 "shops at renewal" takes any MFQ quote in 5 years: 313,077 of 386,673 quotes are more than a year from any due date. R3.O1 writes the policy's **current** lapsed status at every renewal date: 446,153 of 613,403 rows renewed on time that year. MTA documents and uploads are linked to the policy at any time	The join was to the policy or customer, with no date range around the cycle the event belongs to	**Read the design:** every step belongs to one cycle or one sale. Join on the cycle's key and its date range. **Test the output:** pick 5 customers with several cycles and read their events in order

Bug	Example and evidence	Why it got in	What should have been done
One event per policy for 5 years (P15)	M3 "customer accepts" uses `GROUP BY PolicyCode`: 217,044 rows for 345,273 Motor MTA postings, so a second MTA vanishes. The same holds in C1a, D1, CL1a and arrears	Grouping removed the duplicates a join produced, and with them the real repeats	**Test the output:** compare the step's row count with the source's event count. An MTA step should have about one row per MTA posting
Month bucketing (P14)	Most Motor renewal cells use `GROUP BY policy, date_trunc('MONTH', ts)` with `max(ts)`. Counts then depend on month boundaries, and the latest time is kept, not the first	A month was a convenient way to collapse repeats	**Ask Vijay** what one event is: one per occurrence (cycle, transaction or call), at its first time unless the step says otherwise (X2)
Fan-out repeats (P6)	A sale joined to payments by guid fans out, up to 81 copies (A4a). DocumentStatus rows repeat up to 146 times (A6.B1.3). TYR emails repeat up to 9 times per guid and minute (A2.E1)	A one-to-many join was not expected	**Test the output:** count exact duplicate rows per step. Anything above zero needs an explanation
Mixed policy key (P13)	Some renewal steps key on last year's PolicyCode (R0, R1a, R3a, R5, R6) and others on this year's TyPolicyCode (R2.E1, R4, R3b). 23% of Motor renewals change code, so those steps never meet in one episode	Each cell took whichever code its source carried	**Check the data:** how often does the code change at renewal? Agree one key per journey, and carry both codes on the cycle
Wrong renewal base	Home reads `tbl_fact_policy_renewals` with PolicyOfferNum = 1 and PolicyRetNum = 1, both unreliable since the renewal process changed, with no end date. Van has three different bases, and one of them holds all history (VR5 has 36,332 rows before the window)	Each journey built its own base	**Ask Vijay** for one trusted cycle source for all products (H1, V6), built once and shared

Bug	Example and evidence	Why it got in	What should have been done
Outcomes for renewals not yet due	"Lapsed" means no sale yet, so 149,531 lapse events are dated after the data date (to 2027-08). R2.B1.O3 "buys elsewhere" includes renewals not yet due	The rule did not check that the cycle was over	**Test the output:** no event can be later than the data date. A lapse exists only once due + 7 days has passed
Lapse rules inverted or always true	HR3.B1.1 needs a renewal after the due date, so lapses with no renewal at all are left out. VR3.B1.O1 "renewed inside the window" is "this year's renewal date is later than last year's", which every renewal meets (71,115 rows)	The rule was written without trying it on a known lapse and a known renewal	**Test the output:** run the step on 3 hand-picked policies (renewed, renewed late, lapsed) and check each lands where it should

Journey steps

These are bugs in single steps. The full list is in `out/e1_findings.md`. The examples below were chosen because each teaches something. Most come from a step being written from its name alone, without the flow sheet's text, the source's real content or anyone who knows the process.

Steps that measure something else (P9). The biggest group. In every case the step's name and its filter disagree.

Journey · step	What the cell does	Why it is wrong	What should have been done
Motor Acq · A1 "quote initiated"	Counts every MFQ guid	A retrieve creates a new guid, so 1,261,684 retrieves count as new quotes. 1,483,509 guids never reached step 1	**Ask Vijay** how a quote starts and how a retrieve shows in MFQ

Journey · step	What the cell does	Why it is wrong	What should have been done
Motor Acq · A5.F1 "payment failed"	PaymentType not 0 or 5	Type 1 means started with no response (119k): abandoned, not failed. Only type 4 is a failure	**Ask Vijay** what the codes mean (now R-3). Never infer a code's meaning from its value
Motor Acq · A7.F1 "certificate non-arrival"	The DAY 20 document chase	A document chase is not a certificate failing to arrive	**Read the design:** the flow sheet says what the signal is, or that there is none
Motor Ren · R3b "renews on an inbound call"	Every renewal posting, any channel	241,733 of 374,778 R3b policies also renewed online	**Check the data:** split the result by channel before naming it
Motor Ren · R3c "renews on an outbound call"	Channel = 'a) IB'	That is the inbound code. Outbound is 'c) OB'	**Test the output:** the step's channel mix should be outbound
Motor Canx · C2 to C5	NB document chases, joined by client code to anyone in the cancellations table	Only 2,471 of 17,479 C2 policies were themselves cancelled	**Read the design:** a cancellation step is anchored on the cancellation
Motor Doc Req · D1, D2	Kept a call only when a Relay document was emailed within 2 days	D1 is selected on its own outcome, and the "documents" are bulk renewal and NB packs. Agents send requested documents from their own mail, which is not captured	**Ask Vijay** how the process really runs. The only data is the Documents_Out call and its wrap-up code (D-E1.5)
Motor Arrears · ARR.1, ARR.3	The Chase 1 SMS as "missed payment", the Chase 5 SMS as "loan defaults"	A chase is not the event it chases. The loan file has the real status changes (6 > 5 > 4 > 3)	**Ask Vijay** which source holds the event itself. The loan file `dlk.cda_loanse` was there

Journey · step	What the cell does	Why it is wrong	What should have been done
Home Acq · HA1.F1	Quotes whose email got NO TYR send	The design asks for retrieve sends to quotes that never got a price: the logic is inverted	**Read the design** sentence by sentence against the SQL
Home Acq · HA3.F1 "no price returned"	Any quote without a premium	It also takes HA2.F1's drop-offs, which never asked for a price	**Test the output:** two friction steps of one journey should not share quotes

Copy-paste and mirrored code (P2, P4). Thirty-five findings come from code copied from another step or product without its meaning being re-checked.

Journey · step	What happened	What should have been done
Motor Acq · A1.E1	Copy of A1 ("no service grid data, assume same as A1"): 5.36M invented events	If there is no data, the step has no signal. **Ask Vijay**, never fill with a copy
Motor Acq · A6.B1.F1	Mirrored from Van: kept PolicyType = 'Renewals' and a literal end date of 2026-07-31, so renewal policies count as acquisition friction	**Check the data:** re-read every filter after copying, and justify each one for the new product
Motor Ren · R6.F1 "non-arrival"	Mirrored from Home: "suitability statement sent" (887,587 rows)	**Read the design** for the target product, not the source one
Home Acq · HA7.B1.5, Home Ren · HR5, HR6	Motor certificate steps copied to Home, which has no certificate: built, zero rows	**Test the output:** a step that returns no rows is a question, not a result
Many	Identical pairs: A7.B1 = A7.B1.1, R0.F1 = R0.B1, R3.B1 = R3.B1.1, R6.B1 = .1 = .2, M4 = M4a, HA7.B1.1 = HA7.B1.2, HA6 = HA6.B1.4	**Test the output:** compare row counts across a journey's steps. Two identical counts are a bug until shown otherwise

Journey · step	What happened	What should have been done
Many	Descriptions copied from other steps: "Documents requested" on A7, A6.B1.5 and A6.B1.O1; A2's text on A2.F1	Write each step's description from the flow sheet

SQL that cannot work (P11).

Step	Bug	What should have been done
M1.F1	`CustomerPhoneNumber NOT IN (SELECT ...)`, and the list holds a NULL, so it can never return a row, though 117,465 abandoned MTA calls exist	**Test the output:** zero rows means stop and look
R3.B1.2.F1	`PolicyCode NOT IN` the same set: 0 rows	As above
A6, A6.F1	`'2ndCar_Cert_Status' = 'O'` compares a string literal, so it is never true	Quote column names with backticks, not quotes. Test each filter on its own
stg.JulyPolicyState	Two missing commas: MTAGrossNum is dropped and MTCGrossNum holds its value; ReportingSaleDate holds RenewGrossNum	**Test the output:** profile a builder's columns against its source

Calls picked wrongly (P10). A0 and HA0 take a call only when its **last** session's queue is the sales queue, so transfers in or out are lost. A3.E1 looks for an outbound queue on inbound conversations only. A0.F1 and HA0.F1 take abandoned calls with wait > 0, not the agreed rule (wait of 600 s or more, or no agent answered). R2b, R3b.F1, M1 and HR2b calls carry no policy or customer, so they cannot join an episode. **Check the data:** read a few full conversations in `genesys_session_summary` to see how sessions, transfers and directions are laid out. Then **ask Vijay** for the attribution rule.

Built against the design, and not built where it had data. A7.B1.3, M.V1 to M.V4 and VA7 are built though the flow sheet marks them "no signal". Meanwhile 40 of Motor Acquisition's 73 designed steps were never built, and VR0, where the Van renewal journey starts, is missing. **Read the design:** build what it asks, flag where you disagree, and say why.

Habits for next time

Ten minutes of checks per step would have stopped most of the 122 findings. Use this list on every step of the new build. Tick each item before the step is handed over.

Before writing the step

- [ ] Read the flow sheet's text for the step, not just its name. If it says "no signal", don't build it, or raise it.
- [ ] Name the source of record, and check it is built by this notebook or by an agreed upstream table (R-1, R-2).
- [ ] Ask Vijay whenever a code, status, channel or flag has to be interpreted, or two tables look alike. Write the answer in the decision log.
- [ ] Profile the source: row count, min and max of each date, top values of each key (look for `GDPR`, blanks, NULLs), and nulls in the join column for this product.
- [ ] Follow one real customer, policy or call through the source by hand.
- [ ] Decide the step's grain (one per call, transaction, cycle or sale) and the real timestamp that goes with it.

While writing

- [ ] Take the window from the build config. No literal dates.
- [ ] Join events to their cycle or sale with a date range, not only to the policy or customer.
- [ ] Fill every key that can be derived (customer, policy, quote, event reference), and only with keys of the right type (R-6).
- [ ] When copying from another step or product, re-justify every filter and rewrite the description.
- [ ] Make the step re-runnable: delete its own rows, or rebuild its table, before inserting.

Before handing over

- [ ] Row count against the source's own event count (for example, MTA rows against MTA postings).
- [ ] Zero exact duplicate rows.
- [ ] Min and max timestamp inside the window and not after the data date.
- [ ] Zero rows, or the same count as another step, is explained in writing.
- [ ] Read 10 sample rows, every column.
- [ ] Read 3 to 5 customers' full episodes in time order: is the story believable?
- [ ] Run the step twice: the count does not change.

When to ask, and when to check. Data answers what is there: which column is filled, whether a time exists, how many rows repeat. Vijay answers what it means: which source is trusted, what a code stands for, how the process runs. Most of the bugs above came from guessing the second kind of answer from the first kind of evidence, or from not looking at all.